Assignment No: 10

Title: Develop a webpage to fetch and display data from an API using Fetch and JSON.

Aim: To develop a webpage that requests data from a remote REST API using the Fetch API, parses the JSON response, and renders the result, including correct handling of asynchronous code and of failures.

Objectives:

1. To understand the client-server request-response model and the HTTP methods.
2. To use the Fetch API to send GET and POST requests.
3. To understand Promises and the async / await syntax.
4. To serialize a JavaScript object into JSON with JSON.stringify and parse a response with response.json().
5. To set request headers and send a JSON request body.
6. To handle network errors, non-200 responses and malformed payloads.
7. To render the parsed data into the page and show loading and error states.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- Node.js (v18 or later) and npm
- An API key for the Google Gemini / Cloud Vision APIs (stored in a .env file)

Theory:

1. APIs and JSON

An Application Programming Interface (API) lets one program request data from another over HTTP. A REST API exposes resources at URLs and uses HTTP verbs: GET to read, POST to create, PUT or PATCH to update and DELETE to remove.

JSON (JavaScript Object Notation) is the usual exchange format. It is text, so it must be converted at both ends: JSON.stringify(obj) turns an object into a JSON string for sending, and JSON.parse(text) or response.json() turns a received string back into an object.

2. The Fetch API

fetch(url, options) starts an HTTP request and immediately returns a Promise that resolves to a Response object. The body is not downloaded yet, so a second await is needed to read it as .json(), .text() or .blob().

GET request:

```
const res  = await fetch(url);
const data = await res.json();
```

POST request with a JSON body:

```
const res = await fetch(url, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ query: "Modi script" })
});
const data = await res.json();
```

3. Promises and async / await

A Promise represents a value that is not available yet. It is pending, then either fulfilled or rejected. async marks a function as returning a Promise, and await pauses inside that function until a Promise settles, which lets asynchronous code be written in a linear, readable style.

Style	Example
Chained	fetch(url).then(r => r.json()).then(d => render(d))
async / await	const d = await (await fetch(url)).json(); render(d);

4. Error Handling

- fetch only rejects on a network failure; an HTTP 404 or 500 still resolves, so response.ok or response.status must be checked explicitly.
- A try / catch block around await catches both the network rejection and a JSON parse failure.
- An API may answer HTTP 200 with an error object in the body, so the payload itself must be validated before it is used.

5. Keeping API Keys out of Source Control

A key must never be hard-coded. In a Vite project the key is placed in a .env file as VITE_GEMINI_KEY=... and read as import.meta.env.VITE_GEMINI_KEY, while .env is listed in .gitignore. A VITE_ prefixed variable is still bundled into the client, so in production the request should be proxied through a server.

Course Project Implementation:

Course-Project Mapping: Fully available in the course project. All three of LipiSutra's network modules are pure Fetch + JSON implementations. Submit src/services/translate.js as the compact example with full error handling, src/services/vision.js for parsing a deeply nested JSON response, and src/services/gemini.js for a two-stage POST pipeline with a structured JSON response contract.

Concept	File	Evidence
fetch + POST + headers	src/services/translate.js	method, Content-Type header,

		JSON.stringify body
await response.json()	all three service modules	const data = await response.json()
try / catch error handling	src/services/translate.js	catch returns a readable message instead of crashing the UI
API error inside a 200 body	src/services/gemini.js	if (!data.candidates) throw new Error(data.error?.message)
Parsing nested JSON	src/services/vision.js	responses[0].fullTextAnnotation.pages > blocks > paragraphs > words
Rendering the parsed result	src/components/PublicView.jsx	tabs bound to result.summary, result.transcript, result.locations
Key kept in .env	all service modules	import.meta.env.VITE_GEMINI_KEY

Program:

Project file - src/services/translate.js (POST + JSON + try/catch)

```
const GEMINI_KEY = import.meta.env.VITE_GEMINI_KEY;   // read from .env, never hard-coded

export async function translateText(text, targetLang) {
  try {
    const prompt =
      `Translate the following text into the ISO-639-1 language code '${targetLang}'.` +
      `Return strictly ONLY the translated text, nothing else. Text to translate:\n\n${text}`;

    const response = await fetch(
      `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI_KEY}`,
      {
        method: "POST",                                    // HTTP verb
        headers: { "Content-Type": "application/json" },    // request header
        body: JSON.stringify({                             // object -> JSON text
          contents: [{ parts: [{ text: prompt }] }]
        })
      }
    );

    const data = await response.json();                    // JSON text -> object

    // the API can answer 200 OK with an error object inside the body
    if (data.error) throw new Error(data.error.message);

    return data.candidates[0].content.parts[0].text;

  } catch (err) {
    console.error("Translation API Error:", err);
    return "Error: Could not translate the text. " + err.message;
  }
}
```

Project file - src/services/gemini.js (two-stage POST pipeline, abridged)

```
const GEMINI_KEY = import.meta.env.VITE_GEMINI_KEY;
```

```
async function callEngine(base64Image, prompt, temperature) {
  const response = await fetch(
    `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
flash:generateContent?key=${GEMINI_KEY}`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        contents: [{
          parts: [
            { inlineData: { mimeType: "image/jpeg", data: base64Image } },
            { text: prompt }
          ]
        }],
        generationConfig: {
          temperature,
          maxOutputTokens: 8192,
          responseMimeType: "application/json"   // ask the API for strict JSON
        }
      })
    }
  );

  const data = await response.json();

  if (!data.candidates) {
    console.error("Gemini API error:", JSON.stringify(data));
    throw new Error(data.error?.message || "Gemini API returned no result");
  }

  const raw = data.candidates[0].content.parts[0].text;
  return JSON.parse(raw.trim());                  // second parse: the model's own
JSON
}

export async function analyzeDocument(base64Image) {
  // STAGE 1 : a cheap classification request
  const gatekeeperPrompt =
    `Analyze the physical condition of this manuscript. Is it torn, severely degraded,
` +
    `blurred, or physically missing text? Return ONLY exactly ` +
    `{"is_damaged": true} or {"is_damaged": false}.`;

  const gatekeeper = await callEngine(base64Image, gatekeeperPrompt, 0.0);
  const isDamaged  = gatekeeper.is_damaged === true;

  // STAGE 2 : route to the appropriate engine based on stage 1
  const prompt = isDamaged ? RECONSTRUCTOR_PROMPT : LITERAL_OCR_PROMPT;
  const payload = await callEngine(base64Image, prompt, isDamaged ? 0.7 : 0.1);

  // defensive defaults so the UI never maps over undefined
  payload.aiPredictions        = payload.aiPredictions        || [];
  payload.reconstructionNeeded = payload.reconstructionNeeded || [];
  payload.locations            = payload.locations            || [];

  return payload;
}
```

Project file - src/services/vision.js (parsing a deeply nested JSON response)

```
const GOOGLE_KEY = import.meta.env.VITE_VISION_KEY;

export async function getConfidenceScores(base64Image) {
  const response = await fetch(
    `https://vision.googleapis.com/v1/images:annotate?key=${GOOGLE_KEY}`,
    {
      method: "POST",
      headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({
        requests: [{
          image:    { content: base64Image },
          features: [{ type: "DOCUMENT_TEXT_DETECTION" }]
        }]
      })
    }
  );

  const data  = await response.json();
  const words = [];

  // optional chaining guards every level of the nested payload
  const pages = data.responses[0]?.fullTextAnnotation?.pages || [];

  pages.forEach(page =>
    page.blocks?.forEach(block =>
      block.paragraphs?.forEach(para =>
        para.words?.forEach(word => {
          const text = word.symbols?.map(s => s.text).join("") || "";
          words.push({ word: text, confidence: word.confidence || 0 });
        })
      )
    )
  );

  return words;   // [{ word: "...", confidence: 0.97 }, ...]
}
```

Project file - src/components/UploadSection.jsx (calling the API and rendering states)

```
async function handleAnalyze() {
  if (!base64) return;

  setLoading(true);                                  // LOADING state
  setProgress(0);

  const hash       = await generateHash(base64);
  const cachedData = await checkExistingHash(hash);  // avoid a paid call if possible

  if (cachedData) {
    onResult(cachedData);                            // SUCCESS state, from cache
    setProgress(100);
    setLoading(false);
    return;
  }

  try {
    const geminiResult = await analyzeDocument(base64);   // the Fetch call
    onResult(geminiResult);                                // SUCCESS state
    setProgress(100);
    setLoading(false);
    saveDocument(geminiResult, "public", hash)
      .catch(e => console.warn("Firebase intercept failed", e));
  } catch (err) {
    alert("Error analyzing document: " + err.message);    // ERROR state
    setProgress(100);
    setLoading(false);
  }
}
```

Supporting file - .env (never committed; .env.example is committed instead)

```
VITE_GEMINI_KEY=your_gemini_api_key_here
VITE_VISION_KEY=your_cloud_vision_api_key_here
VITE_MAPS_KEY=your_google_maps_api_key_here
```

Output:

- Clicking "Decode & Analyze Artifact" issues a POST to the Gemini endpoint; the Network tab shows the request with Content-Type: application/json and a JSON request body.
- The response is parsed with response.json() and the resulting object populates the Summary, Transcript and Origin tabs of the page.
- Re-uploading the same image returns in about 50 ms from the Firestore cache instead of issuing a second paid API call, as the console log "[CACHE_CHECK] HIT!" confirms.
- Disconnecting the network and retrying is caught by the try/catch block and surfaces the message "Error analyzing document: Failed to fetch" instead of a blank screen.
- Supplying an invalid API key makes the API answer HTTP 200 with an error object; the explicit ! data.candidates check converts it into a readable error message.

Conclusion:

In this assignment, we developed a webpage that fetches and displays data from a remote API using the Fetch API and JSON. We sent POST requests with headers and a JSON.stringify body, awaited response.json(), and rendered the parsed object into the page. We handled all three outcomes explicitly - loading, success and failure - including the case where the API returns HTTP 200 with an error object in the body, and we kept the API keys out of source control in a .env file.

References:

8. Mozilla Developer Network (MDN) Web Docs - JavaScript Guide, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide
9. W3Schools - JavaScript Tutorial, https://www.w3schools.com/js/
10. ECMAScript Language Specification, https://tc39.es/ecma262/
11. MDN Web Docs - Using the Fetch API, https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch
12. MDN Web Docs - JSON, https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON